

Adora — Plan B: Neural-Field Synthesis / Inversion (MLP + Fourier features)

Extension plan · Ivan Milic · 2026-08-28 · 3/3

Idea. Represent the atmosphere as a coordinate-based *neural field*: a small MLP $f_\theta(\gamma(\mathbf{x}))$ with Fourier-feature encoding γ maps position $\mathbf{x} = (x, y, z)$ to the 8 physical parameters. The existing differentiable RT operator maps that atmosphere to Stokes profiles, and we fit θ to observations — a PINN-style / neural-field inversion that **replaces the fixed nodes of `nodes.py` with a continuous, resolution-independent, smoothly regularised representation**. It slots straight into Adora because gradients already flow spectrum $\rightarrow$ RT $\rightarrow$ atmosphere $\rightarrow \theta$, so `optax` (already in `iterate_adam.py`) trains it directly.

- **Encoding.** $\gamma(\mathbf{x}) = [\sin(2\pi\mathbf{B}\mathbf{x}), \cos(2\pi\mathbf{B}\mathbf{x})]$ with Gaussian random frequencies $\mathbf{B}$ (Tancik et al. 2020), tuned per axis — z needs a finer bandwidth than x, y . This defeats the MLP "spectral bias" so sharp photospheric T /velocity gradients are representable.
- **Network & output transforms.** Small MLP ($\sim 3\text{--}5$ layers, width 64–256, SiLU/tanh) $\rightarrow$ 8 outputs, each passed through a physical map (softplus/exp for densities & v_{turb} , scaled sigmoid for T , tanh for v_z , bounded B, γ_B, χ_B) — reuse the bounds already in `nodes.py` so predictions stay physical.
- **Loss.** Sample the field on each column's z -grid $\rightarrow$ `lte_polarised_rt` $\rightarrow$ Stokes; minimise χ^2 data misfit + physics/regularisation terms (hydrostatic/EOS residual, smoothness, optionally $\nabla \cdot \mathbf{B} = 0$) — the "PINN regularisation" from the README TODO.
- **Training & validation.** Adam then L-BFGS (`optax`), mini-batched over pixels/ λ , `jit+vmmap`. Prove on a single pixel, then a 2D map (spatial coherence via shared θ is the payoff); invert a synthetic FALC/MURaM cube and compare recovered atmosphere, fit quality and speed against the `nodes.py` node inversion and the ground truth.
- **Risks.** Parameter magnitudes differ by orders (normalise in/out); NN can run `fp32` while opacities stay `fp64` (mixed precision); ill-posedness needs the regularisers/priors above.
- **Deliverable.** `neural_field.py` (encoding + MLP + transforms) and `invert_field.py` (training loop), sharing the RT forward model with Plan A.